Universidad Diego Portales



Escuela de Informática y Telecomunicaciones

Torneo de Komikan Concurrente

Tarea 2 – Sistemas Operativos

Integrantes
Alexander Bravo
Isaías Vásquez

Repositorio: `https://github.com/IsaV1/Tarea2-S.O`

Noviembre de 2026

Índice

1. Resumen **1**

2. Diseño del esquema de sincronización **1**
2.1. Arquitectura 1
2.2. Exclusión mutua en los movimientos 1
2.3. Orden de locks y reentrada 1
2.4. Emparejamiento y mesas 1

3. Resiliencia **1**
3.1. Interrupción (SIGINT) 1
3.2. Abandono de jugadores (simulado) 2
3.3. Interpretación y alcance 2

4. Análisis de riesgos de concurrencia **2**

5. Evaluación de las decisiones de diseño **2**

6. Pruebas realizadas **2**

7. Conclusión **2**

Torneo de Komikan Concurrente

Tarea 2 – Sistemas Operativos

Alexander Bravo Isaías Vásquez

1. Resumen

Este informe describe el diseño de sincronización de un torneo de Komikan concurrente: N jugadores simulados con hilos POSIX se emparejan, ocupan hasta M mesas y juegan partidas en tiempo real, sin turnos. Se detallan las primitivas empleadas, los riesgos de concurrencia (deadlock, livelock y starvation) y su mitigación, la evaluación del diseño y las pruebas realizadas. Las instrucciones de compilación y el formato de `config.txt` se describen en el `README.md` del repositorio: `https://github.com/IsaV1/Tarea2-S.O`

2. Diseño del esquema de sincronización

2.1 Arquitectura

El torneo se simula con hilos POSIX dentro de un proceso (C++17, sin red). `main` crea los hilos de jugadores (que forman parejas al llegar a una cola compartida y simulan un posible abandono, sección 3.2) y atiende las parejas a medida que se forman, sin esperar a que todas estén listas. Por cada pareja crea una `Partida` (con su propio `Tablero`, por lo que partidas distintas no comparten estado) y un hilo árbitro. El árbitro espera una de las M mesas, coloca las fichas, inicia los 13 hilos de pieza (1 Puma y 12 Perros) y espera su finalización con `join`. Cada pieza duerme un retardo aleatorio en [`minMs`, `maxMs`] y solicita su movimiento a `Partida::moverPorId`, que lo valida, lo ejecuta y registra en consola (partida, pieza, origen, destino, captura y matriz del tablero). Finalmente, el árbitro informa el resultado y libera la mesa.

Tabla 1: Recursos compartidos y primitivas empleadas.

Recurso compartido	Primitiva	Función
Tablero (casillas, capturas)	`lock_tablero` (uno por partida)	lecturas y escrituras de casillas
Partida (iniciada, terminada, ganador, abandono)	`lock_partida` (uno por partida)	transiciones de estado consistentes
ColaJugadores (esperando, parejas, asignaciones)	`lock_cola` + `cond_parejas`	emparejamiento y aviso de partida asignada
Mesas (máximo M activas)	`sem_t` con valor inicial M	espera pasiva de mesa libre
Consola	`lock_cout`	movimiento y tablero impresos sin mezclarse
Orden de cierre (Ctrl+C)	`atomic<bool>` lock-free	bandera única escrita por el manejador

2.2 Exclusión mutua en los movimientos

Para que dos fichas no ocupen la misma casilla, `Tablero::moverPorId` hace en una sola sección crítica: (i) buscar la posición actual de la pieza por su identificador, (ii) validar adyacencia o captura y que el destino esté vacío, y (iii) escribir el movimiento (incluida la eliminación del perro capturado). La primera ficha que adquiere el lock ocupa la casilla. Como la posición se busca dentro del lock, un hilo nunca usa un origen obsoleto ni mueve una pieza ajena. Los destinos candidatos se calculan antes (`listarDestinos`) solo como sugerencia: se revalidan dentro del lock y, si el tablero cambió, el intento se rechaza y se reintenta tras el siguiente retardo.

2.3 Orden de locks y reentrada

Las funciones `validar*SinLock` evitan volver a adquirir un mutex no recursivo. Las rutas que usan varios mutex respetan un orden compatible con la jerarquía `lock_partida` → `lock_cout` → `lock_tablero`; otras rutas toman solo algunos de ellos. No se adquieren en orden inverso.

2.4 Emparejamiento y mesas

Cada jugador toma `lock_cola`: si hay otro esperando forma pareja (el primero juega Puma y el segundo Perros) y hace `pthread_cond_broadcast`; si no, espera su asignación. `main` intenta crear primero los hilos de jugadores y luego va consumiendo las parejas disponibles o esperando nuevas con `pthread_cond_timedwait` (1 s, para poder revisar la bandera de Ctrl+C); no necesita que todas las parejas se hayan formado. El árbitro hace `sem_wait` antes de colocar las piezas y `sem_post` al terminar o cancelar: nunca hay más de M partidas jugando y las parejas sobrantes permanecen bloqueadas en el semáforo (espera pasiva), sin consumir CPU. Ciclo de vida: el árbitro hace `join` de sus 13 piezas, y `main` de jugadores y árbitros antes de liberar cada `Partida` y destruir mutex, variable de condición y semáforo.

3. Resiliencia

3.1 Interrupción (SIGINT)

El manejador solo escribe la bandera `detenerServidor`. `main` deja de crear jugadores y partidas, cancela las partidas existentes (`cancelarPartida` las marca terminadas sin ganador) y despierta a los jugadores que esperan. Las piezas dividen su retardo en tramos de hasta 100 ms y, entre ellos, revisan SIGINT y el estado de la partida, evitando esperar todo el retardo antes de terminar. Los árbitros que aún esperaban mesa la revisan al obtenerla, la devuelven con `sem_post` y terminan. Todos los hilos se unen antes de liberar los recursos.

3.2 Abandono de jugadores (simulado)

Cada jugador dispone de un hilo propio que, una vez iniciada su partida, evalúa su estado cada 100 ms; en cada evaluación existe una probabilidad aproximada de 1 % de que abandone, calculada con `rand_r` y una semilla individual. Si ocurre, `Partida::abandonarPartida(id)` actualiza el estado bajo `lock_partida`: registra al jugador que abandona, marca la partida como terminada y asigna la victoria al jugador contrario. Como `terminado` se comprueba dentro de la misma sección crítica, un abandono y una victoria normal simultáneos no pueden producir un segundo ganador. Los hilos de pieza finalizan al observar la partida terminada, y el árbitro registra el abandono y el ganador antes de liberar la mesa.

3.3 Interpretación y alcance

Los jugadores son hilos simulados dentro de un mismo programa, sin personas reales ni clientes conectados; por ello no existe una acción externa de «salir de la partida», y el escenario equivalente es un fallo o una salida inesperada de un jugador. Se representó mediante un abandono probabilístico, en lugar de provocar fallos reales de hilos, que resultan más difíciles de detectar de forma segura en este diseño. La implementación simula una salida inesperada; no detecta automáticamente un fallo real ni el bloqueo de un hilo.

4. Análisis de riesgos de concurrencia

Condiciones de carrera: Casillas y capturas: un mutex por tablero y validación + escritura atómicas. Estado de la partida: `lock_partida`. Cola y asignaciones: `lock_cola`. Mensajes: `lock_cout`.

Deadlock: Se previene con un orden consistente de locks, con las funciones `SinLock` y evitando esperas bloqueantes con el mutex de partida o tablero tomado: `sem_wait` y el retardo de las piezas se ejecutan sin esos locks, y `pthread_cond_wait` libera `lock_cola`. No se afirma que el riesgo sea nulo; en las ejecuciones repetidas no se observaron bloqueos (sección 6).

Livelock: Un destino elegido fuera del lock puede dejar de ser válido; el intento falla, no modifica el tablero y el hilo reintenta tras un retardo aleatorio dentro de [`minMs`, `maxMs`] (por defecto 50–300 ms), que desincroniza a las fichas en conflicto. No hay una garantía formal de progreso inmediato.

Starvation: `sem_t` no garantiza orden FIFO y no se implementó una cola justa de mesas. En las pruebas con $M = 1$ las partidas obtuvieron mesa en orden de formación; con $M = 2$ hubo una inversión (la partida 4 antes que la 3). Todas las partidas terminaron. Dentro de una partida tampoco hay equidad formal entre fichas: el reparto de oportunidades es probabilístico.

5. Evaluación de las decisiones de diseño

Un mutex por tablero: Baja complejidad y fácil de razonar. Serializa los movimientos de una partida, pero cada pieza mueve una vez por retardo configurable [`minMs`, `maxMs`] (por defecto 50–300 ms), así que la contención es baja, y las partidas distintas avanzan en paralelo hasta M. Un lock por casilla exigiría un orden global de adquisición y más complejidad sin una ganancia que lo justifique.

Un árbitro por pareja, creado al formarse: Evita esperar a que lleguen todos, pero deja hasta $N/2$ hilos bloqueados en `sem_wait` y sin garantía de orden (ver Starvation). Una cola FIFO con un solo hilo servidor daría orden a cambio de más código.

Aleatoriedad: Cada hilo usa `rand_r` con su propia semilla, sin generador global compartido. La misma semilla no reproduce la partida exacta, porque el planificador cambia el orden de llegada de los movimientos.

Alcance: Sin sockets ni usuarios externos; cada jugador participa en una sola partida.

6. Pruebas realizadas

Se compiló con `g++ -Wall -Wextra -std=c++17 -pthread` sin advertencias y se comprobó el rechazo de parámetros inválidos (N impar, $N < 2$, $M < 1$) con código 1. Se ejecutó con distintas cantidades de jugadores y mesas, verificando que se respeta el máximo M. Se probó la lectura de configuraciones válidas (el `config.txt` entregado usa retardos de 100–200 ms) e inválidas (incompletas, fuera de rango o con tiempos inválidos), en cuyo caso se informa el error y se usan los valores por defecto. También se comprobó la finalización normal y por abandono de las partidas y la cancelación por Ctrl+C en distintas fases (al arrancar, con partidas activas y con parejas esperando mesa). En los escenarios ejecutados no se observaron bloqueos ni errores de sincronización visibles.

7. Conclusión

El diseño busca un equilibrio entre simplicidad y seguridad concurrente mediante primitivas POSIX y secciones críticas atómicas. La serialización por tablero limita el paralelismo interno de cada partida y la asignación de mesas carece de equidad formal. En las pruebas realizadas, el sistema respetó el límite de mesas y permitió finalizar o cancelar las partidas sin observar anomalías críticas. El diseño cumple el alcance de la simulación, aunque no ofrece garantías formales de equidad entre hilos.